

Practical:5

Doubly Linked List and Circular Linked List Operations

Practical Aim

To implement core doubly linked list and circular linked list operations and understand how bidirectional and circular linking change the way nodes are inserted, deleted, and traversed.

Problem 1

A music player maintains a playlist where songs can be added to the beginning, added to the end, or inserted right after a specific song that is currently playing. When a song is removed, only the first song in the playlist is dropped. At any point the player can count how many songs are in the playlist and display them from first to last. Given a sequence of such operations, implement all of them on a doubly linked playlist and print the result after each operation. Describe how your insertion after a given song works. What extra step does a doubly linked list require here that a singly linked list did not? What happens if the given song is not found in the playlist?

CODE:

```
package prac5;

public class problem1 {

    class Node {
        String song;
        Node prev, next;

        Node(String song) {
            this.song = song;
        }
    }

    Node head, tail;
    int count = 0;
```

```
void addFirst(String song) {
    Node newNode = new Node(song);

    if (head == null)
        head = tail = newNode;
    else {
        newNode.next = head;
        head.prev = newNode;
        head = newNode;
    }

    count++;
    display();
}

void addLast(String song) {
    Node newNode = new Node(song);

    if (tail == null)
        head = tail = newNode;
    else {
        tail.next = newNode;
        newNode.prev = tail;
        tail = newNode;
    }

    count++;
    display();
}

void insertAfter(String song, String newSong) {
    Node temp = head;

    while (temp != null && !temp.song.equals(song))
        temp = temp.next;

    if (temp == null) {
        System.out.println("Song not found");
        display();
        return;
    }

    Node newNode = new Node(newSong);

    newNode.next = temp.next;
    newNode.prev = temp;
```

```
        if (temp.next != null)
            temp.next.prev = newNode;
        else
            tail = newNode;

        temp.next = newNode;

        count++;
        display();
    }

    void removeFirst() {
        if (head == null) {
            System.out.println("Playlist is empty");
            return;
        }

        head = head.next;

        if (head == null)
            tail = null;
        else
            head.prev = null;

        count--;
        display();
    }

    void display() {
        Node temp = head;

        System.out.print("Playlist: ");

        while (temp != null) {
            System.out.print(temp.song + " ");
            temp = temp.next;
        }

        System.out.println("\nCount: " + count);
        System.out.println();
    }

    public static void main(String[] args) {

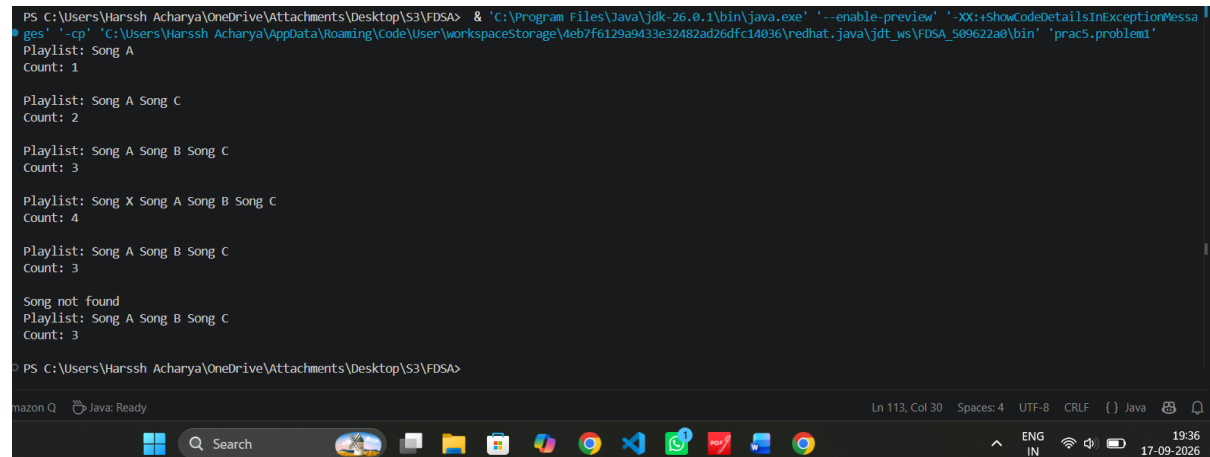
        problem1 p = new problem1();
```

```

        p.addFirst("Song A");
        p.addLast("Song C");
        p.insertAfter("Song A", "Song B");
        p.addFirst("Song X");
        p.removeFirst();
        p.insertAfter("Song Z", "Song D");
    }
}

```

OUTPUT:



```

PS C:\Users\Harssh Acharya\OneDrive\Attachments\Desktop\S3\FDSA> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Harssh Acharya\AppData\Roaming\Code\User\workspaceStorage\4eb7f6129a9433e32482ad26dfc14036\redhat.java\jdt_ws\FDSA_509622a0\bin' 'prac5.problem1'
Playlist: Song A
Count: 1

Playlist: Song A Song C
Count: 2

Playlist: Song A Song B Song C
Count: 3

Playlist: Song X Song A Song B Song C
Count: 4

Playlist: Song A Song B Song C
Count: 3

Song not found
Playlist: Song A Song B Song C
Count: 3

PS C:\Users\Harssh Acharya\OneDrive\Attachments\Desktop\S3\FDSA>

```

Problem 2

A group of students are sitting in a circle playing a passing game. A token starts at the first student and is passed around the circle one student at a time. Students can join the circle at any position and leave at any time, and the circle must remain unbroken after every join or leave. Given a sequence of join, leave, and display operations, implement them on both a singly circular and a doubly circular linked list and print the current circle after each operation. Describe how your solution ensures the circle stays unbroken after a student leaves. What is the one case where breaking the circle is most likely to happen as a bug, and how did you handle it?

CODE:

```

package prac5;

public class problem2 {
    static class SinglyCircular {
        class Node {
            String name;
            Node next;

```

```
Node(String name) {
    this.name = name;
}

Node head = null;
Node tail = null;

void add(String name) {
    Node newNode = new Node(name);

    if (head == null) {
        head = tail = newNode;
        newNode.next = head;
    } else {
        newNode.next = head;
        tail.next = newNode;
        tail = newNode;
    }
    display();
}

void remove(String name) {
    if (head == null)
        return;

    Node current = head;
    Node previous = tail;

    do {
        if (current.name.equals(name)) {
            if (current == head && current == tail) {
                head = tail = null;
            } else {
                previous.next = current.next;

                if (current == head)
                    head = current.next;

                if (current == tail)
                    tail = previous;
            }
            break;
        }
        previous = current;
        current = current.next;
    }
```

```
        } while (current != head);

        display();
    }

    void display() {
        if (head == null) {
            System.out.println("Circle is empty");
            return;
        }

        Node temp = head;

        System.out.print("Singly Circle: ");

        do {
            System.out.print(temp.name + " ");
            temp = temp.next;
        } while (temp != head);

        System.out.println();
    }
}

static class DoublyCircular {
    class Node {
        String name;
        Node prev, next;

        Node(String name) {
            this.name = name;
        }
    }

    Node head = null;

    void add(String name) {
        Node newNode = new Node(name);

        if (head == null) {
            head = newNode;
            newNode.next = head;
            newNode.prev = head;
        } else {
            Node tail = head.prev;

            newNode.next = head;
```

```
        newNode.prev = tail;
        tail.next = newNode;
        head.prev = newNode;
    }

    display();
}

void remove(String name) {
    if (head == null)
        return;

    Node current = head;

    do {
        if (current.name.equals(name)) {
            if (current.next == current) {
                head = null;
            } else {
                current.prev.next = current.next;
                current.next.prev = current.prev;

                if (current == head)
                    head = current.next;
            }
            break;
        }
        current = current.next;
    } while (current != head);

    display();
}

void display() {
    if (head == null) {
        System.out.println("Circle is empty");
        return;
    }

    Node temp = head;

    System.out.print("Doubly Circle: ");

    do {
        System.out.print(temp.name + " ");
        temp = temp.next;
    }
```

```
    } while (temp != head);

    System.out.println();
}
}

public static void main(String[] args) {

    SinglyCircular s = new SinglyCircular();
    System.out.println("SINGLY CIRCULAR LINKED LIST");

    s.add("A");
    s.add("B");
    s.add("C");
    s.remove("B");
    s.add("D");
    s.remove("A");

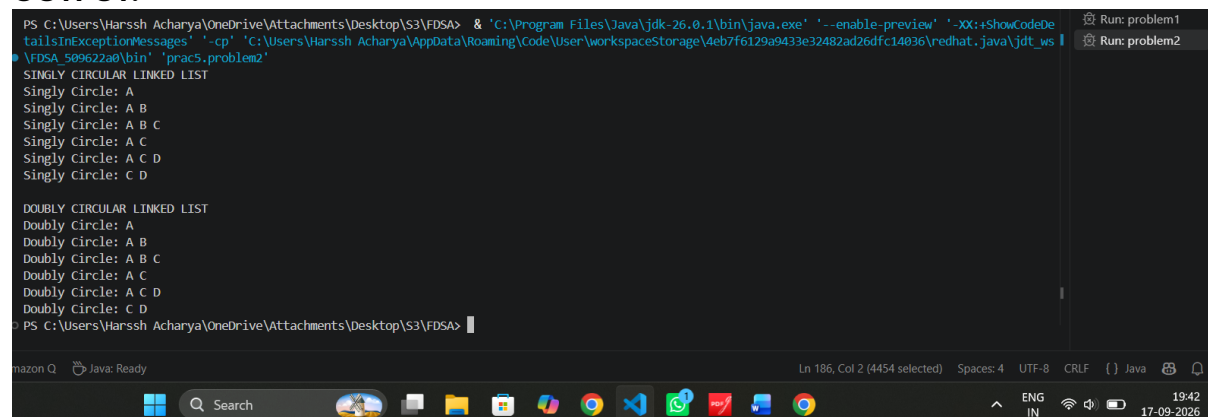
    System.out.println();

    DoublyCircular d = new DoublyCircular();

    System.out.println("DOUBLY CIRCULAR LINKED LIST");

    d.add("A");
    d.add("B");
    d.add("C");
    d.remove("B");
    d.add("D");
    d.remove("A");
}
}
```

OUTPUT:



```
PS C:\Users\Harsh Acharya\OneDrive\Attachments\Desktop\S3\FDSA> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDe
tailsInExceptionMessages' '-cp' 'C:\Users\Harsh Acharya\AppData\Roaming\Code\User\workspaceStorage\4eb7f6129a9433e32482ad26dfc14036\redhat.java\jdt_ws
\FDSA_509622a0\bin' 'prac5.problem2'
SINGLY CIRCULAR LINKED LIST
Singly Circle: A
Singly Circle: A B
Singly Circle: A B C
Singly Circle: A C
Singly Circle: A C D
Singly Circle: C D

DOUBLY CIRCULAR LINKED LIST
Doubly Circle: A
Doubly Circle: A B
Doubly Circle: A B C
Doubly Circle: A C
Doubly Circle: A C D
Doubly Circle: C D
PS C:\Users\Harsh Acharya\OneDrive\Attachments\Desktop\S3\FDSA>
```

GitHub link: <https://github.com/haarshaacharya/FDSA/tree/main/prac5>